

Data Model & Data Dictionary

dashAI — Saksi mata digital yang netral & tertandatangan

dashAI

2026-06-21

Daftar Isi

1	Pendahuluan	2
2	Model Konseptual	3
2.1	Prinsip desain data	3
2.2	Bukan basis data relasional	4
2.3	Entitas tingkat tinggi	4
3	Entitas Logis	5
3.1	Tipe enumerasi pendukung	5
3.2	ViolationEvent	5
3.3	EvidencePayload	6
3.4	SealedEvidence	6
3.5	Citation	6
3.6	ViolationMeta	7
3.7	RoadContext	7
3.8	GeoPoint	7
4	Diagram Entity-Relationship	7
5	Kamus Data (Data Dictionary)	9
5.1	ViolationEvent	9
5.2	EvidencePayload	11
5.3	SealedEvidence	13
6		13
6.1	Citation	13
6.2	ViolationMeta	14
6.3	RoadContext	15
6.4	GeoPoint	16
6.5	MediaHashes	17
6.6	LegalSnapshot	17
6.7	VerificationResult	17
7		18
7.1	Pemetaan pelanggaran → pasal (snapshot)	18
8	Kanonikalisasi & Hashing Payload yang Ditandatangani	20
8.1	Mengapa kanonikalisasi diperlukan	20
8.2	Algoritma kanonikalisasi	20

8.3	Hashing	20
8.4	Penandatanganan (seal)	21
8.5	Verifikasi	21
8.6	Apa yang dibuktikan dan tidak dibuktikan	21
9	Skema Penyimpanan Lokal (IndexedDB)	22
9.1	Database & object store	22
9.2	Operasi yang tersedia	22
9.3	Degradasi tanpa IndexedDB	22
9.4	Apa yang TIDAK disimpan di server	22
9.5	State sesi (non-persisten)	23
10	Retensi & Penghapusan Data	23
10.1	Filosofi: kontrol di tangan pemilik	23
10.2	Retensi per lokasi penyimpanan	24
10.3	Penghapusan yang tersedia	24
10.4	Pertimbangan privasi & data pribadi	24

1 Pendahuluan

Dokumen ini mendefinisikan **model data** dashAI secara menyeluruh: entitas logis, hubungan antarentitas, kamus data per-field, mekanisme kanonikalisasi dan hashing payload yang ditandatangani, skema penyimpanan lokal (IndexedDB), serta kebijakan retensi dan penghapusan data.

dashAI adalah aplikasi web (Next.js 16 App Router, React 19, TypeScript mode strict, Tailwind CSS v4) yang berjalan terutama **di browser/ponsel** dan di-deploy pada Vercel (<https://dashai-mu.vercel.app>). Aplikasi ini mendeteksi pelanggaran lalu lintas secara real-time di sisi klien (TensorFlow.js coco-ssd + IoU tracker + MediaPipe face/pose + Tesseract.js OCR pelat), mengaitkan setiap pelanggaran ke pasal **UU No. 22 Tahun 2009 tentang Lalu Lintas dan Angkutan Jalan (LLAJ)** dari knowledge base terverifikasi, dan **menyegel bukti secara kriptografis** dengan Ed25519 di server.

Seluruh definisi dalam dokumen ini di-*ground* langsung pada kode sumber. Sumber kebenaran (source of truth) untuk model tipe adalah:

Artefak	Berkas	Peran dalam model data
Tipe domain inti	lib/evidence/types.ts	Definisi ViolationEvent, EvidencePayload, SealedEvidence, GeoPoint, RoadContext, MediaHashes, LegalSnapshot, dll.
Katalog pelanggaran	lib/legal/catalog.ts	ViolationMeta + taksonomi VIOLATION_CATALOG (12 entri).
Knowledge base hukum	lib/legal/citations.ts	Citation + CITATIONS (12 entri, verifikasi 3-voter).
Konstruksi payload	lib/evidence/payload.ts	buildPayload() — bagaimana payload yang ditandatangani dibentuk.
Kanonikalisasi & hash	lib/crypto/canonical.ts	Serialisasi deterministik + SHA-256.
Penandatanganan	lib/crypto/sign.ts, lib/crypto/keys.ts	Ed25519 sign + manajemen kunci server.
Verifikasi	lib/crypto/verify.ts	Verifikasi hash + tanda tangan.
Penyimpanan lokal	lib/evidence/store.ts	Skema IndexedDB (idb).
State sesi	lib/state/session.ts	Store sesi runtime (zustand).
Konteks jalan	lib/geo/osm.ts, lib/geo/location.ts	Sumber RoadContext dan GeoPoint.

Catatan integritas. dashAI bersifat **tamper-evident**, bukan **tamper-proof**. Tanda tangan membuktikan bahwa isi laporan tidak berubah satu byte pun sejak disegel server pada waktu tertera; ia **tidak** membuktikan bahwa kamera benar-benar menyaksikan peristiwa fisik di dunia nyata. Lihat bagian *Kanonikalisasi & Hashing Payload yang Ditandatangani*.

2 Model Konseptual

2.1 Prinsip desain data

Model data dashAI dibangun di atas lima prinsip yang menentukan setiap keputusan struktur:

1. **Local-first.** Bukti milik pemilik disimpan lebih dulu di perangkatnya sendiri (IndexedDB). Server hanya pernah melihat sebuah payload ketika pengguna **secara eksplisit** menyegel/

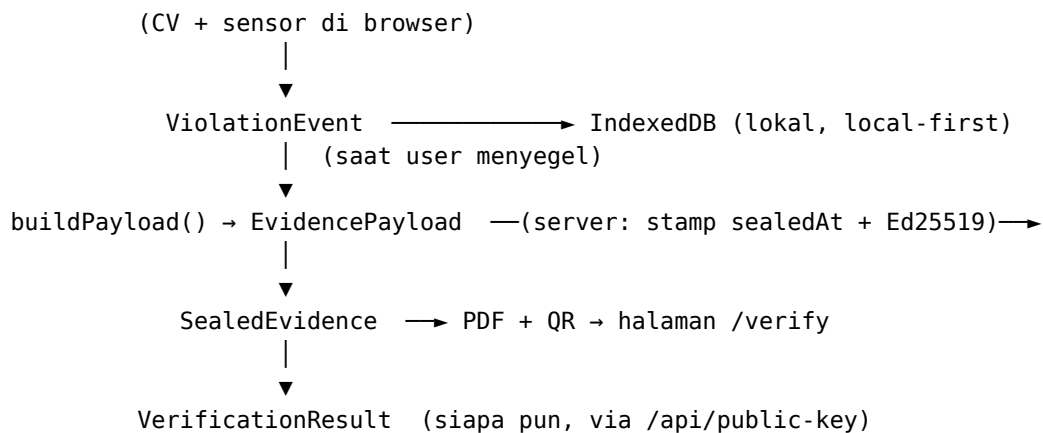
melaporkan suatu peristiwa. Tidak ada basis data terpusat yang menyimpan kejadian pengguna.

2. **Batas kriptografis tunggal yang eksplisit.** Hanya satu struktur yang ditandatangani: `EvidencePayload`. Segala sesuatu di luarnya (media blob, overlay UI, scene demo) bersifat presentasional dan **tidak memiliki bobot pembuktian**. Komentar header `lib/evidence/types.ts` menyatakan ini secara tegas: *“The cryptographic boundary is EvidencePayload.”*
3. **Privacy-by-design.** Wajah dideteksi, **bukan** dikenali (face detection, not recognition); blur-by-default di UI; koordinat GPS dibulatkan ke 4 desimal (~11 m) sebelum dikirim ke server pihak ketiga Overpass; payload yang ditandatangani meng-*commit* gambar lewat hash (`MediaHashes`), bukan dengan menyertakan megabita base64. Selaras dengan semangat UU No. 27 Tahun 2022 (Pelindungan Data Pribadi).
4. **Dua subjek (dual-subject).** Setiap pelanggaran berdimensi subject: “other” (penindakan terhadap pihak lain — laporan tilang) atau “self” (perlindungan pemilik — bukti meringankan + self-coaching).
5. **Kejujuran pengukuran.** Model membedakan pengukuran **akurat** dari **estimasi**. Kecepatan ego (`egoSpeedKmh`) berasal dari GPS dan akurat; kecepatan kendaraan lain (`otherSpeedKmh`) adalah **estimasi** dengan rentang error. Perbedaan ini dikodekan sebagai dua field terpisah, bukan satu.

2.2 Bukan basis data relasional

Penting digarisbawahi: dashAI **tidak** memiliki basis data server-side untuk data kejadian. “Model data” di sini adalah **model tipe** (TypeScript) yang mengalir melalui pipeline bukti, dengan satu-satunya penyimpanan persisten di sisi klien (`IndexedDB` dashai, object store events). Server bersifat *stateless* terhadap kejadian: ia menerima payload, menyetel/memverifikasi/ merender, lalu mengembalikan hasilnya tanpa menyimpan apa pun. Bahkan laporan PDF bersifat **self-contained** — seluruh envelope tersegel di-*encode* ke dalam QR/URL halaman `/verify`, sehingga verifikasi tidak memerlukan lookup basis data.

2.3 Entitas tingkat tinggi



Entitas referensi (tidak mengalir, melainkan dirujuk):

- **ViolationMeta** — metadata taksonomi pelanggaran (VIOLATION_CATALOG).
- **Citation** — entri knowledge base hukum (CITATIONS), sumber dari LegalSnapshot yang ditanam ke payload pada saat seal.

3 Entitas Logis

Bagian ini mendeskripsikan setiap entitas logis utama. Tipe ditulis persis seperti pada kode sumber (lib/evidence/types.ts, lib/legal/catalog.ts, lib/legal/citations.ts).

3.1 Tipe enumerasi pendukung

Sebelum entitas, beberapa tipe union yang dipakai berulang:

Tipe	Nilai	Sumber
Subject	"other" "self"	lib/evidence/types.ts
ViolationKey	12 kunci (lihat di bawah)	lib/evidence/types.ts
SceneType	"street-day" "street-night" "intersection" "highway" "alley"	lib/evidence/types.ts
ReportKind	"tilang" "kecelakaan" "coaching"	lib/evidence/types.ts
DetectionTier	"core" "secondary" "cabin"	lib/legal/catalog.ts
CitationConfidence	"high" "medium" "low" "pending"	lib/legal/citations.ts

ViolationKey (12 nilai, terkunci ke katalog dan knowledge base): lawan-arus, tanpa-helm, penumpang-tanpa-helm, terobos-lampu-merah, langgar-marka, boncengan-lebih, melebihi-kecepatan, tanpa-sabuk, main-hp, tanpa-plat, tanpa-lampu-malam, motor-lampu-siang.

3.2 ViolationEvent

Representasi **client-side** dari satu pelanggaran yang sudah dikonfirmasi oleh engine. Inilah entitas yang disimpan ke IndexedDB. Field media (frame, faceCrop, plateCrop) menyimpan object URL / data URL selama berada di memori; setelah disegel, gambar dirujuk lewat hash di EvidencePayload.mediaHashes, bukan di-embed ke tanda tangan.

ViolationEvent adalah superset presentasional: ia memuat informasi untuk UI (media blob, scene, notes, bbox) **plus** subset field yang akan diangkat ke EvidencePayload. Hanya subset itulah yang ditandatangani.

Karakteristik kunci:

- id — identitas stabil event (dihasilkan klien, dipakai sebagai keyPath IndexedDB).
- confidence — keyakinan engine (0..1) bahwa pelanggaran benar-benar terjadi.

- `egoSpeedKmh` (akurat, GPS) vs `otherSpeedKmh` (**estimasi**) — dua field terpisah, sesuai prinsip kejujuran pengukuran.
- `sealed + seal` — status segel dan envelope tersegel (jika ada).
- `demo` — menandai data demonstrasi terkurasi yang **tidak pernah** boleh diperlakukan sebagai bukti nyata.

3.3 EvidencePayload

Batas kriptografis. Field-field persis inilah — diserialisasi secara kanonik (`lib/crypto/canonical.ts`) — yang ditandatangani server. Tanda tangan membuktikan payload ini disegel oleh instance `dashAI` pada `sealedAt` dan tidak berubah sejak itu; ia **tidak** membuktikan kamera menyaksikan realitas fisik.

`EvidencePayload` dibangun oleh `buildPayload()` (`lib/evidence/payload.ts`) dari sebuah `ViolationEvent` plus:

- `sealedAt` — **dicap server** (waktu otoritatif), tidak pernah dipercaya dari klien. Di `/api/seal` nilai ini berasal dari `Date.now()` server.
- `legal` (`LegalSnapshot`) — di-*resolve* dari knowledge base (`citationFor(event.violation)`) pada saat seal, sehingga laporan mencerminkan versi pasal yang berlaku.
- `mediaHashes` (`MediaHashes`) — SHA-256 dari byte media, agar payload meng-*commit* citra tanpa membengkok.
- `device` — metadata perangkat opsional (`userAgent`, `platform`).
- `schema` — selalu literal `"dashai.evidence.v1"` (versi skema, memungkinkan evolusi terversi).

3.4 SealedEvidence

Amplop (envelope) bertanda tangan yang dikembalikan `/api/seal` dan ditanam ke dalam laporan. Memuat payload lengkap, algoritma ("`Ed25519`"), `publicKeyId` (mengikat ke kunci verifikasi), `payloadHash` (hex SHA-256 dari payload kanonik), `signature`, dan `sealedAt`.

`SealedEvidence` adalah unit yang berpindah end-to-end: dari `/api/seal` → disimpan di `ViolationEvent.seal` → di-render ke PDF (`/api/report`) → di-*encode* `base64url` ke URL `/verify?d=... + QR` → diverifikasi oleh siapa pun.

Catatan field signature. Tipe (`SealedEvidence.signature`) berkomentar "`base64url Ed25519 signature`", sedangkan implementasi `sealPayload()` saat ini mengembalikan `bytesToHex(sig)` (hex). Verifikasi (`verifySealed()`) konsisten membaca `hexToBytes(sealed.signature)`. Jadi **encoding aktual yang ditulis dan dibaca adalah hex**; kamus data di bawah mencatat keduanya agar tidak menyesatkan.

3.5 Citation

Entri **knowledge base hukum** untuk UU 22/2009 (LLAJ). Di-*generate* dari output terverifikasi workflow riset Phase-0 `dashAI` (verifikasi adversarial **3-voter** per-sitasi terhadap sumber resmi/terpercaya Indonesia — `hukumonline`, `korlantas.polri.go.id`, `dishub`, `peraturan.bpk.go.id`, dll). Berkasnya tidak boleh diedit manual; ia di-regenerate dari workflow riset.

Tiap entri memuat teks lengkap pasal (bunyi), ringkasan ancaman pidana (sanksi), denda maksimum (dendaMaxRupiah), kurungan maksimum (kurunganMax), pasal-pasal terkait (relatedArticles), tingkat keyakinan (confidence), status verifikasi (verified), dan daftar sumber (sources).

CITATIONS adalah Record<ViolationKey, Citation> — tepat 12 entri, satu per ViolationKey, seluruhnya confidence: "high" dan verified: true.

3.6 ViolationMeta

Metadata taksonomi per pelanggaran (VIOLATION_CATALOG, juga Record<ViolationKey, ViolationMeta>, 12 entri). Memuat label dwibahasa (labelId/labelEn), penjelasan singkat (blurb), subjek yang berlaku (subjects), tier deteksi (tier), dasar deteksi CV/sensor (detectionBasis), apakah butuh kamera kabin (requiresCabinCam), dan tingkat keparahan (severity, 1..3).

ViolationMeta adalah entitas referensi: ia mengendalikan UI dan menjelaskan *bagaimana* sebuah pelanggaran dideteksi, tetapi tidak ikut ditandatangani. Mesin aturan bersifat *pluggable* — menambah pelanggaran baru berarti menambah satu entri ViolationMeta, satu Citation, dan satu rule detector.

3.7 RoadContext

Konteks jalan yang di-*resolve* dari OpenStreetMap (Overpass API, tanpa API key) untuk titik GPS saat ini. Memuat nama jalan (name), apakah satu-arah (oneway), arah sah perjalanan sebagai bearing way OSM dalam derajat (bearingDeg), dan batas kecepatan (maxspeedKmh).

RoadContext adalah paruh *map-grounded* dari deteksi lawan-arah dan referensi legal untuk speeding subjek "self". Hasil di-*cache* per sel grid ~11 m dan *degrade* mulus ke null pada kegagalan jaringan agar pipeline live tidak pernah memblokir.

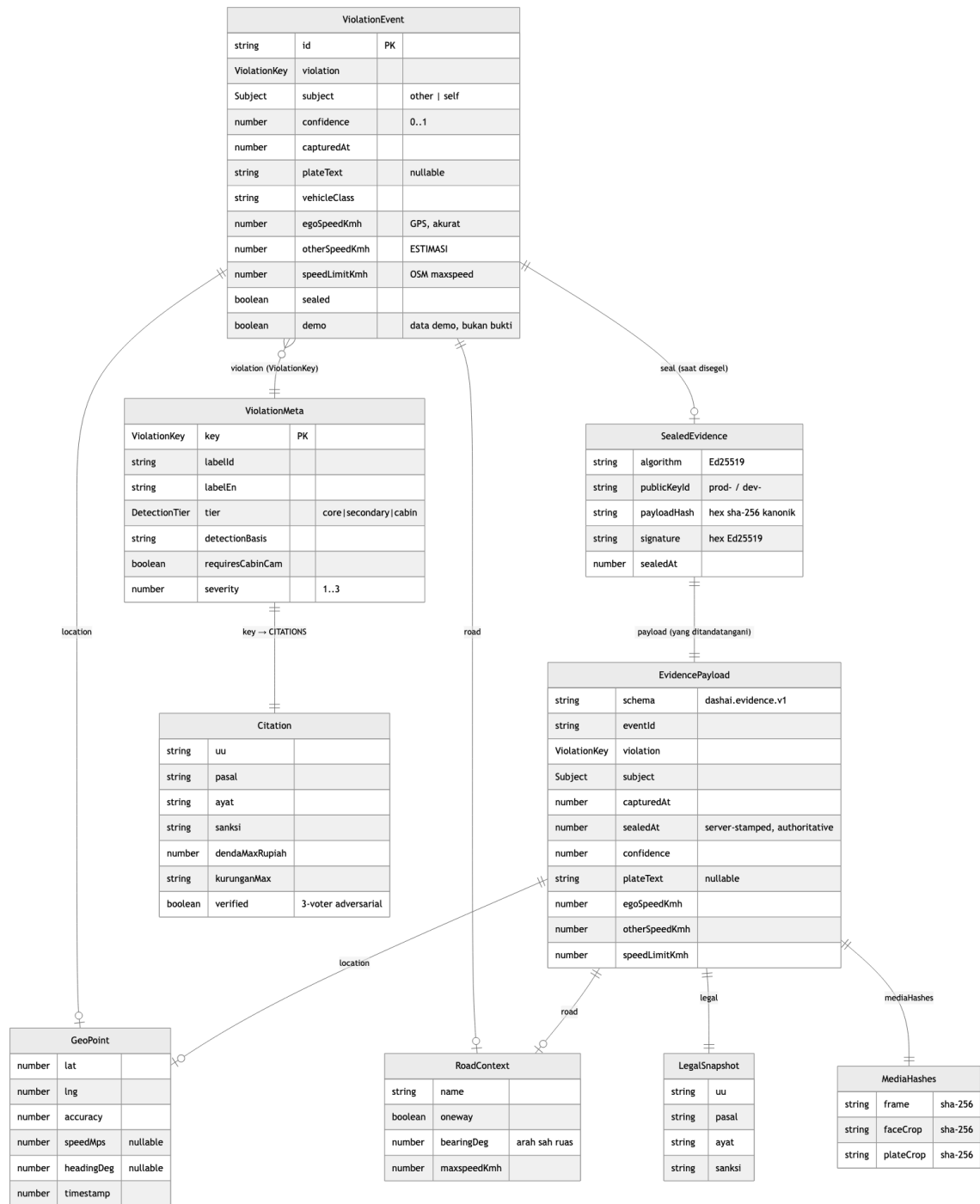
3.8 GeoPoint

Satu sampel GPS. lat/lng (derajat), accuracy (meter, opsional), speedMps dan headingDeg datang langsung dari Geolocation API dan **boleh null** pada perangkat yang tidak melaporkannya, serta timestamp (epoch ms).

GeoPoint muncul di dua entitas: di ViolationEvent.location dan, setelah seal, di EvidencePayload.location.

4 Diagram Entity-Relationship

Diagram berikut memetakan hubungan antarentitas model (di-*ground* persis pada lib/evidence/types.ts, lib/legal/citations.ts, dan lib/legal/catalog.ts). Catatan: ini adalah **model tipe**, bukan skema basis data relasional — IndexedDB hanya menyimpan ViolationEvent.



Gambar 1: Entity-Relationship Diagram model data dashAI

Ringkasan kardinalitas:

Relasi		Kardinalitas	Makna
ViolationEvent SealedEvidence	→	1 : 0..1	seal ada hanya setelah event disegel.
ViolationEvent GeoPoint	→	1 : 0..1	location (opsional/null).
ViolationEvent RoadContext	→	1 : 0..1	road (opsional/null).
ViolationEvent ViolationMeta	→	many : 1	violation (ViolationKey) memetakan ke katalog.
SealedEvidence EvidencePayload	→	1 : 1	payload — objek yang ditandatangani.
EvidencePayload GeoPoint	→	1 : 0..1	location.
EvidencePayload RoadContext	→	1 : 0..1	road.
EvidencePayload LegalSnapshot	→	1 : 1	legal (snapshot sitasi).
EvidencePayload MediaHashes	→	1 : 1	mediaHashes (bisa {} kosong).
ViolationMeta → Citation		1 : 1	key → entri CITATIONS.

5 Kamus Data (Data Dictionary)

Setiap field dicatat lengkap: tipe, makna, batasan/constraint, dan sumber. Tipe ditulis persis seperti deklarasi TypeScript. Tanda ? menandakan field opsional; | null menandakan dapat bernilai null.

5.1 ViolationEvent

Sumber: lib/evidence/types.ts. Disimpan di IndexedDB store events.

Field	Type	Makna	Batasan / Constraint	Sumber nilai
id	string	Identitas unik event; IndexedDB.	Wajib, unik.	Dihasilkan klien (uid()).
violation	ViolationKey	Jenis pelanggaran.	Salah satu dari 12 kunci.	Engine aturan.
subject	Subject	Subjek pelanggaran.	"other" "self".	Engine aturan.
confidence	number	Keyakinan pelanggaran terjadi.	0..1.	Engine; di- <i>clamp</i> server saat seal.
capturedAt	number	Waktu pengambilan (epoch ms).	Wajib, > 0.	Klien (waktu frame).
frame	string?	Frame teranotasi penuh.	Object/data URL; client-only.	Canvas render.
faceCrop	string?	Crop wajah (selalu di-blur di UI).	Object/data URL; client-only.	CV (face detection).
plateCrop	string?	Crop region pelat.	Object/data URL; client-only.	CV (plate).
plateText	string null?	Teks pelat hasil OCR.	Bebas; null bila gagal/tidak ada.	Tesseract.js OCR.
vehicleClass	string?	Kelas kendaraan COCO.	mis. car, motorcycle.	Detektor coco-ssd.
egoSpeedKmh	number null?	Kecepatan kendaraan pemilik.	Akurat (GPS) ; km/jam.	GeoPoint.speedMps × 3.6.
otherSpeedKmh	number null?	Kecepatan kendaraan lain.	ESTIMASI (ada error).	Estimator (skala bbox + flow).
speedLimitKmh	number null?	Batas kecepatan ruas.	km/jam.	OSM maxspeed.
location	GeoPoint null?	Lokasi GPS.	Lihat GeoPoint.	Geolocation API.
road	RoadContext null?	Konteks jalan. Untuk jalan biasa, dapat lebih (normalisasi).	Lihat RoadContext.	OSM Overpass. Dapsid.no.tracker.

Subtipe BBox (`lib/evidence/types.ts`): `x`, `y`, `w`, `h` — semua number, ternormalisasi 0..1 terhadap dimensi frame.

5.2 EvidencePayload

Sumber: `lib/evidence/types.ts`; dibentuk oleh `buildPayload()` (`lib/evidence/payload.ts`).

Inilah objek yang ditandatangani.

Field	Type	Makna	Batasan / Constraint	Sumber nilai
schema	"dashai.evidence.v1"	Versi skema payload.	Literal tetap.	buildPayload().
eventId	string	Mengacu ke ViolationEvent.id	Wajib.	Event.
violation	ViolationKey	Jenis pelanggaran.	12 kunci; divalidasi in CITATIONS.	Event.
subject	Subject	Subjek.	"other" "self"; divalidasi server.	Event.
capturedAt	number	Waktu pengambilan klien (epoch ms).	typeof === "number".	Event.
sealedAt	number	Waktu seal server (otoritatif).	Date.now() server.	/api/seal.
confidence	number	Keyakinan engine.	Di-clamp ke 0..1 di server.	Event (dinormalisasi).
vehicleClass	string?	Kelas kendaraan.	—	Event.
plateText	string null?	\ Teks pelat.	Default null (?? null).	Event.
egoSpeedKmh	number null?	\ Kecepatan ego (akurat).	Default null.	Event.
otherSpeedKmh	number null?	\ Kecepatan kendaraan lain (estimasi).	Default null.	Event.
speedLimitKmh	number null?	\ Batas kecepatan.	Default null.	Event.
location	GeoPoint null?	\ Lokasi.	Default null.	Event.
road	RoadContext null?	\ Konteks jalan.	Default null.	Event.
mediaHashes	MediaHashes	Hash SHA-256 media.	Default {} bila tak ada.	/api/seal body.
legal	LegalSnapshot	Snapshot sitasi pasal.	Di-resolve saat seal.	validationFor(violation).
device	{ userAgent?: string; platform?: string }?	Metadata perangkat.	Opsional.	/api/seal body.

5.3 SealedEvidence

Sumber: `lib/evidence/types.ts`; dihasilkan `sealPayload()` (`lib/crypto/sign.ts`).

Field	Tipe	Makna	Batasan / Constraint	Sumber nilai
<code>payload</code>	<code>EvidencePayload</code>	Payload yang ditandatangani.	Lihat tabel di atas.	<code>buildPayload()</code> .
<code>algorithm</code>	<code>"Ed25519"</code>	Algoritma tanda tangan.	Literal tetap.	<code>sealPayload()</code> .
<code>publicKeyId</code>	<code>string</code>	ID kunci publik (prod-/dev- + 16 hex).	Mengikat ke kunci verifikasi.	<code>getServerKey()</code> .
<code>payloadHash</code>	<code>string</code>	Hex SHA-256 payload kanonik.	64 karakter hex	<code>hex.sha256Hex0fString()</code> .
<code>signature</code>	<code>string</code>	Tanda tangan Ed25519 atas byte kanonik.	Hex (tipe menyebutkan base64url; implementasi & verifikasi hex).	<code>ed.signAsync()</code> .
<code>sealedAt</code>	<code>number</code>	Salinan <code>payload.sealedAt</code> .	6 <code>payload.sealedAt</code> .	<code>sealPayload()</code> .

6.1 Citation

Sumber: `lib/legal/citations.ts` (`CITATIONS: Record<ViolationKey, Citation>`).

Field	Tipe	Makna	Batasan / Constraint	Sumber nilai
uu	string	Nama undang-undang.	UU 22/2009 (LLAJ).	Riset Phase-0.
pasal	string	Pasal.	mis. "Pasal 287".	Riset Phase-0.
ayat	string?	Ayat.	mis. "ayat (1)".	Riset Phase-0.
bunyi	string	Teks lengkap pasal.	Verbatim sumber.	Riset Phase-0.
sanksi	string	Ringkasan ancaman pidana.	—	Riset Phase-0.
dendaMaxRupiah	number?	Denda maksimum (rupiah).	mis. 500000.	Riset Phase-0.
kurunganMax	string?	Kurungan maksimum.	mis. "2 bulan".	Riset Phase-0.
relatedArticles	string[]?	Pasal-pasal terkait.	—	Riset Phase-0.
confidence	CitationConfidence	Tingkat keyakinan sitasi.	high/medium/low/pending.	Riset Phase-0.
verified	boolean?	Lolos cek 3-voter adversarial.	true untuk 12/12.	Riset Phase-0.
sources	string[]?	URL sumber.	Sumber resmi/terpercaya.	Riset Phase-0.

Subset Citation yang diangkat ke LegalSnapshot (dan ikut ditandatangani): uu, pasal, ayat, sanksi.

6.2 ViolationMeta

Sumber: `lib/legal/catalog.ts` (VIOLATION_CATALOG: `Record<ViolationKey, ViolationMeta>`).

Field	Tipe	Makna	Batasan / Constraint	Sumber nilai
key	ViolationKey	Kunci pelanggaran.	Salah satu dari 12.	Katalog.
labelId	string	Label Bahasa Indonesia.	—	Katalog.
labelEn	string	Label Inggris.	—	Katalog.
blurb	string	Deskripsi satu baris perilaku.	—	Katalog.
subjects	Subject[]	Subjek yang berlaku.	Subset {other,self}.	Katalog.
tier	DetectionTier	Tier keandalan deteksi.	core/ secondary/ cabin.	Katalog.
detectionBasis	string	Sinyal CV/ sensor yang dipakai.	—	Katalog.
requiresCabinCam	boolean	Perlu kamera kabin.	—	Katalog.
severity	1 \ 2 \ 3	Keparahan (1 ringan – 3 berat).	1/2/3.	Katalog.

6.3 RoadContext

Sumber: `lib/evidence/types.ts`; di-*resolve* `resolveRoadContext()` (`lib/geo/osm.ts`).

Field	Tipe	Makna	Batasan / Constraint	Sumber nilai
name	string?	Nama jalan.	OSM tag name.	Overpass.
oneway	boolean?	Apakah satu-arah.	true jika oneway $\in \{\text{yes}, \text{true}, 1\}$.	Overpass tag.
bearingDeg	number null?	Arah sah ruas (bearing way OSM).	0..360 derajat; null jika geometri <2 titik.	Dihitung dari geometri way.
maxspeedKmh	number null?	Batas kecepatan.	Angka pertama dari tag maxspeed.	Overpass tag.

6.4 GeoPoint

Sumber: `lib/evidence/types.ts`; dari `watchLocation()` (`lib/geo/location.ts`).

Field	Tipe	Makna	Batasan / Constraint	Sumber nilai
lat	number	Lintang (derajat).	—	Geolocation API.
lng	number	Bujur (derajat).	—	Geolocation API.
accuracy	number?	Akurasi posisi (meter).	≥ 0 .	Geolocation API.
speedMps	number null?	Kecepatan (m/s).	null bila perangkat tak melapor.	Geolocation API.
headingDeg	number null?	Arah hadap (derajat).	null bila tak tersedia.	Geolocation API.
timestamp	number	Waktu sampel (epoch ms).	—	Geolocation API.

Privasi koordinat. Sebelum dikirim ke server Overpass pihak ketiga, `lat`/`lng` dibulatkan ke 4 desimal (`toFixed(4)`, ~11 m), sehingga koordinat presisi penuh tidak pernah keluar perangkat ke pihak ketiga (`lib/geo/osm.ts`).

6.5 MediaHashes

Sumber: `lib/evidence/types.ts`. SHA-256 (hex) dari byte media — agar payload yang ditandatangani meng-*commit* citra tanpa membengkok oleh base64.

Field	Tipe	Makna	Batasan / Constraint	Sumber nilai
frame	string?	SHA-256 frame penuh.	64 hex.	Klien (hash byte media).
faceCrop	string?	SHA-256 crop wajah.	64 hex.	Klien.
plateCrop	string?	SHA-256 crop pelat.	64 hex.	Klien.

6.6 LegalSnapshot

Sumber: `lib/evidence/types.ts`. Snapshot sitasi yang ditanam ke payload pada saat seal (`legal: { uu, pasal, ayat, sanksi }` di `buildPayload()`).

Field	Tipe	Makna	Batasan / Constraint	Sumber nilai
uu	string	Undang-undang.	Dari <code>Citation.uu</code> .	<code>citationFor()</code> .
pasal	string	Pasal.	Dari <code>Citation.pasal</code> .	<code>citationFor()</code> .
ayat	string?	Ayat.	Dari <code>Citation.ayat</code> .	<code>citationFor()</code> .
sanksi	string	Ancaman pidana.	Dari <code>Citation.sanksi</code> .	<code>citationFor()</code> .

6.7 VerificationResult

Sumber: `lib/evidence/types.ts`; dihasilkan `verifySealed()` (`lib/crypto/verify.ts`).

Field	Tipe	Makna	Batasan Constraint	/	Sumber nilai
valid	boolean	Sah	=	—	verifySealed(). hashMatches && signatureValid.
reason	string	Penjelasan hasil (Bahasa Indonesia).		—	verifySealed().
hashMatches	boolean	Hash kanonik cocok.		—	verifySealed().
signatureValid	boolean	Tanda tangan Ed25519 valid.		—	verifySealed().
publicKeyId	string	ID kunci yang dipakai.	7		verifySealed(). sealed.publicKeyId.
checkedAt	number	Waktu verifikasi (epoch ms).		Date.now().	verifySealed().

7.1 Pemetaan pelanggaran → pasal (snapshot)

Tabel berikut adalah pemetaan otoritatif 12 ViolationKey ke pasal UU 22/2009, **persis konsisten** dengan lib/legal/citations.ts. Tier dari lib/legal/catalog.ts.

ViolationKey	Label (ID)	Tier	Subjek	Pasal · Ayat	Denda maks	Kurungan maks
lawan- arus	Melawan arah	core	other, self	Pasal 287 · ayat (1)	Rp500.000	2 bulan
tanpa- helm	Pengendara tanpa helm	core	other	Pasal 291 · ayat (1)	Rp250.000	1 bulan
penumpang- tanpa- helm	Penumpang tanpa helm	secondary	other	Pasal 291 · ayat (2)	Rp250.000	1 bulan
terobos- lampu- merah	Menerobos lampu merah	core	other, self	Pasal 287 · ayat (2)	Rp500.000	2 bulan
langgar- marka	Melanggar marka/ rambu	secondary	other, self	Pasal 287 · ayat (1)	Rp500.000	2 bulan
boncengan- lebih	Boncengan lebih dari satu	core	other	Pasal 292 · ayat (9)	Rp250.000	1 bulan
melebihi- kecepatan	Melebihi batas kecepatan	core	other, self	Pasal 287 · ayat (5)	Rp500.000	2 bulan
tanpa- sabuk	Tanpa sabuk keselamatan	cabin	self	Pasal 289 · ayat (6)	Rp250.000	1 bulan
main-hp	Bermain ponsel saat berkendara	cabin	self	Pasal 283 · ayat (1)	Rp750.000	3 bulan
tanpa- plat	Tanpa pelat nomor sah	secondary	other	Pasal 280 · ayat (1)	Rp500.000	2 bulan
tanpa- lampu- malam	Tanpa lampu pada malam hari	secondary	other, self	Pasal 293 · ayat (1)	Rp250.000	1 bulan
motor- lampu- siang	Motor tanpa lampu di siang hari	secondary	other	Pasal 293 · ayat (2)	Rp100.000	15 hari

Seluruh 12 sitasi berstatus `confidence: "high"` dan `verified: true` (lolos verifikasi adversarial 3-voter terhadap sumber resmi/terpercaya). Deteksi live yang aktif saat ini: lawan-aras (lawan-arus), ngebut diri sendiri (melebihi-kecepatan subjek self), boncengan (boncengan-lebih), dan terobos lampu merah subjek self (terobos-lampu-merah). Sisanya dikatalogkan dan ditampilkan via dataset demo; deteksi penuh (helm/pelat/kabin) membutuhkan model khusus / kamera kabin.

8 Kanonikalisasi & Hashing Payload yang Ditandatangani

8.1 Mengapa kanonikalisasi diperlukan

Tanda tangan Ed25519 dibuat atas **byte**, bukan atas objek. Agar dua payload yang secara semantik identik menghasilkan tanda tangan yang sama — dan agar verifier dapat *recompute* hash dengan hasil identik di browser maupun di server — serialisasi JSON harus **deterministik** (byte-identik). Inilah peran `canonicalize()` di `lib/crypto/canonical.ts`.

8.2 Algoritma kanonikalisasi

`canonicalize(value)` menerapkan `JSON.stringify(sortDeep(value))`, dengan `sortDeep` yang:

1. Untuk **array**: memetakan elemen secara rekursif (urutan dipertahankan).
2. Untuk **objek**: mengurutkan kunci secara leksikografis (`Object.keys().sort()`) dan memproses nilai secara rekursif.
3. **Menghapus** properti bernilai `undefined` (tidak representable di JSON).
4. Nilai primitif dikembalikan apa adanya.

Konsekuensi penting bagi model data:

- **Urutan kunci tidak material** — payload dengan urutan field berbeda tetap menghasilkan byte kanonik identik.
- **undefined dijatuhkan**, sedangkan **null dipertahankan**. Karena itu `buildPayload()` secara sengaja menormalkan field opsional menjadi `null` (`plateText ?? null`, `egoSpeedKmh ?? null`, `dst`.) agar kehadiran field stabil dan tidak bergantung pada `undefined`.
- Tipe angka diserialisasi oleh `JSON.stringify` standar (mis. `epoch ms` sebagai integer). Tidak ada pembulatan tambahan pada tahap kanonikalisasi.

8.3 Hashing

`sha256HexOfString(s)` menghitung SHA-256 (via `@noble/hashes`) atas byte UTF-8 string kanonik, dikembalikan sebagai **hex 64 karakter**. Inilah `SealedEvidence.payloadHash`.

```
canon      = canonicalize(payload)           // string JSON deterministik
payloadHash = sha256(utf8(canon)) → hex      // commit terhadap isi
payload
signature  = Ed25519_sign(utf8(canon), privKey) // tanda tangan atas byte
yang SAMA
```

Penting: **baik hash maupun tanda tangan dihitung atas string kanonik canon yang sama** (`utf8ToBytes(canon)`). Hash bukan bagian dari input tanda tangan; ia adalah commitment independen yang memudahkan deteksi perubahan tanpa perlu verifikasi kunci publik.

8.4 Penandatanganan (seal)

`sealPayload()` (`lib/crypto/sign.ts`, ditandai `import "server-only"`):

1. Ambil kunci server (`getServerKey()`): `priv (seed Ed25519) + keyId`.
2. `canon = canonicalize(payload)`.
3. `payloadHash = sha256HexOfString(canon)`.
4. `sig = ed.signAsync(utf8ToBytes(canon), priv)`.
5. Kembalikan `SealedEvidence { payload, algorithm: "Ed25519", publicKeyId: keyId, payloadHash, signature: bytesToHex(sig), sealedAt: payload.sealedAt }`.

Kunci privat `DASHAI_SIGNING_KEY` adalah **satu-satunya rahasia** dan tidak pernah meninggalkan server. Tanpa `env var` yang valid (hex 64 karakter), dipakai **kunci DEV** tetap dan envelope ditandai `keyId` ber-prefix `dev-` (bukti DEV tidak boleh diperlakukan otoritatif). Dengan `env var` valid, prefix `prod-`. `keyId = prefix + 16 karakter pertama hex kunci publik`.

8.5 Verifikasi

`verifySealed(sealed, publicKeyHex)` (`lib/crypto/verify.ts`) berjalan **identik di browser dan server** dan melakukan dua cek independen:

1. **Cek hash** — `recomputedHash = sha256HexOfString(canonicalize(sealed.payload))`; `hashMatches = recomputedHash === sealed.payloadHash`. Membuktikan payload tidak diedit.
2. **Cek tanda tangan** — `ed.verifyAsync(hexToBytes(signature), utf8ToBytes(canon), hexToBytes(publicKeyHex))`. Membuktikan tanda tangan sah untuk payload itu di bawah kunci tersebut.

`valid = hashMatches && signatureValid`. Pesan reason membedakan: payload utuh & sah, hash tidak cocok (isi diubah setelah disegel), atau tanda tangan tidak valid untuk kunci publik ini. Verifier mengambil kunci publik dari `/api/public-key`, sehingga **verifikasi tidak perlu memercayai UI dashAI**.

8.6 Apa yang dibuktikan dan tidak dibuktikan

- **Dibuktikan:** isi `EvidencePayload` tidak berubah satu byte pun sejak disegel server pada `sealedAt`; mengubah pelat/kecepatan/pasal mana pun akan membuat hash tidak cocok dan tanda tangan gagal → **terdeteksi**.
- **TIDAK dibuktikan:** bahwa kamera menyaksikan peristiwa fisik nyata (frame berasal dari klien; seseorang bisa mengarahkan kamera ke layar). Karena itu `dashAI` bersifat **tamper-evident**, bukan **tamper-proof**. Jalan menuju produksi: device attestation (Play Integrity / App Attest), kontinuitas GPS, stempel waktu server, dan `dashcam` perangkat keras khusus.

9 Skema Penyimpanan Lokal (IndexedDB)

9.1 Database & object store

Sumber: `lib/evidence/store.ts` (memakai pustaka `idb`). `dashAI` mengadopsi pendekatan **local-first**: bukti pemilik disimpan di perangkatnya sendiri; server hanya melihat payload ketika pengguna eksplisit menyegel/melaporkan.

Properti	Nilai
Nama database	<code>dashai</code>
Versi	<code>1</code>
Object store	<code>events</code>
keyPath	<code>id</code> (in-line key dari <code>ViolationEvent.id</code>)
Index	by-time pada <code>capturedAt</code>
Tipe nilai	<code>ViolationEvent</code> (objek penuh)

Pembuatan store pada upgrade (versi 1):

```
db.createObjectStore("events", { keyPath: "id" })
store.createIndex("by-time", "capturedAt")
```

9.2 Operasi yang tersedia

Fungsi	Operasi IDB	Catatan
<code>saveEvent(e)</code>	<code>put("events", e)</code>	Upsert by id.
<code>getAllEvents()</code>	<code>getAllFromIndex("events", "by-time")</code> lalu <code>reverse()</code>	Terbaru lebih dulu (urut menurun <code>capturedAt</code>).
<code>getEvent(id)</code>	<code>get("events", id)</code>	Ambil satu event.
<code>deleteEvent(id)</code>	<code>delete("events", id)</code>	Hapus satu event.
<code>clearEvents()</code>	<code>clear("events")</code>	Kosongkan seluruh store.

9.3 Degradasi tanpa IndexedDB

Seluruh fungsi store mengecek `typeof indexedDB === "undefined"` dan **degrade mulus**: `saveEvent/deleteEvent/clearEvents` menjadi no-op, `getAllEvents` mengembalikan `[]`, `getEvent` mengembalikan `undefined`. Aplikasi tetap berjalan (mis. di lingkungan tanpa IndexedDB), hanya tanpa persistensi lokal.

9.4 Apa yang TIDAK disimpan di server

Tidak ada object store atau tabel server yang menyimpan `ViolationEvent`. Endpoint API bersifat *stateless* terhadap kejadian:

Endpoint	Runtime	Menyimpan data?	Fungsi
POST /api/seal	nodejs	Tidak	Stamp sealedAt, lampirkan sitasi, tandatangani → SealedEvidence.
POST /api/verify	nodejs	Tidak	Verifikasi envelope (konvenien; trust tak bergantung padanya).
GET /api/public-key	nodejs	Tidak	Publikasikan kunci publik (cache 1 jam).
POST /api/report	nodejs	Tidak	Verifikasi → render PDF + QR (envelope di-encode ke URL).

Karena laporan PDF menanamkan envelope tersegel lengkap ke QR/URL /verify? d=<base64url>, verifikasi **self-contained** dan tidak memerlukan lookup basis data — tamper-evidence tanpa state server.

9.5 State sesi (non-persisten)

Selain IndexedDB, ada store sesi **in-memory** (zustand, lib/state/session.ts) yang **tidak** persisten: status, pipeline, fps, detectionCount, events (daftar sesi berjalan), geo, road, egoSpeedKmh. State ini hilang saat halaman dimuat ulang dan bukan bagian dari model data persisten.

10 Retensi & Penghapusan Data

10.1 Filosofi: kontrol di tangan pemilik

Konsisten dengan desain local-first dan privacy-by-design, **pemilik perangkat adalah pengendali data**. Tidak ada penyimpanan server-side kejadian, sehingga tidak ada retensi terpusat untuk dihapus — penghapusan dilakukan di sumber (perangkat pengguna).

10.2 Retensi per lokasi penyimpanan

Lokasi	Apa yang disimpan	Retensi	Cara hapus
IndexedDB (dashai > events)	ViolationEvent (termasuk media blob & seal)	Sampai pengguna menghapus / clear site data. Tidak ada TTL otomatis.	deleteEvent(id), clearEvents(), atau hapus data situs browser.
State sesi (zustand)	Data runtime sesi	Sampai reload / reset().	Otomatis (non-persisten).
Cache OSM (lib/geo/osm.ts)	RoadContext per sel ~11 m	Selama lifetime halaman (Map in-memory).	Otomatis saat reload.
Cache kunci server (lib/crypto/keys.ts)	Deskriptor kunci (cached)	Selama proses server hidup.	Otomatis (memori proses).
Server (seal/verify/report)	— (tidak menyimpan kejadian)	Tidak ada.	Tidak berlaku.
Laporan PDF	Envelope tersegel (di file PDF + QR)	Sesuai keputusan pemilik/penerima file.	Hapus file PDF.

10.3 Penghapusan yang tersedia

- **Hapus satu event:** `deleteEvent(id) → delete("events", id)`.
- **Hapus seluruh event:** `clearEvents() → clear("events")`.
- **Hapus total dari browser:** menghapus *site data* untuk origin dashAI membuang database dashai beserta seluruh media blob.

10.4 Pertimbangan privasi & data pribadi

- **Data pribadi minimal:** dashAI melakukan deteksi wajah, **bukan** pengenalan identitas; wajah dan pelat di-blur di UI; payload meng-*commit* citra lewat hash, bukan menyertakan citra penuh.
- **Koordinat dibulatkan** ke ~11 m sebelum dikirim ke Overpass pihak ketiga.
- **Transfer eksplisit:** server hanya menerima payload ketika pengguna memilih untuk menyegel/melaporkan; tidak ada upload otomatis frame/lokasi.
- Pendekatan ini selaras dengan semangat **UU No. 27 Tahun 2022 (Pelindungan Data Pribadi)**, menjadikan pengguna sebagai pengendali atas datanya sendiri.

Batasan demo. dashAI saat ini adalah **demonstrasi teknologi via kamera ponsel**; laporan yang dihasilkan **bukan** dokumen resmi kepolisian, sitasi bersifat indikatif, dan kecepatan kendaraan lain (`otherSpeedKmh`) adalah **perkiraan**. Data demo (`ViolationEvent.demo === true`) tidak boleh diperlakukan sebagai bukti nyata. Kontrol

retensi produksi (mis. kebijakan hapus terjadwal, ekspor terenripsi) termasuk dalam roadmap perangkat keras khusus pasca-investor.